

CPS2000 Assignment Report

Daniel Vella 0147005L

June 26, 2026

1 Introduction

In this report I will start by delving into how my compiler is structured, focusing on one file at a time, since each file directly reflects the tasks as defined in the assignment specification. Then I will discuss the testing suite by showing the expected outcome of the tests found in the source code, and also how the compiler can be run on any sfile of source code.

2 Compiler Structure

The compiler was implemented in the OCaml language, using dune as the build system. I chose to use this language because I realised when conducting my preliminary research that the project would require a lot of enumeration and comparison, for example to deduce what token a certain string forms. This is something that OCaml handles very elegantly through its variant type definition and `match...with` pattern-matching construct. Also dune allows for the setup and management of the testing suite to be kept separate from the main code implementation, while still being simple to run and update.

Each file discussed in the following subsections implements the required task in the last defined function in the file using all the previous type definitions and helper functions to achieve that.

2.1 Task 1 - lexer.ml

The natural first step of the Lexer is defining the tokens as a new type, allowing me to pattern match an incoming character or string onto a token. Below is an example of part of the type definition, and how it is used in part of the `map_keywords` function:

```
type token =
  (* Structure and Declaration *)
  Extern | Type | Function | Locals | Void | Entry | To |
  (* Instruction *)
  Const | Cast | Un | Bin | Addr_Of | Member_Ptr | Load | Store | Call |
  Jump | Cjump | Return | ...

let map_keywords s =
  match s with
  | "extern" -> Extern | "type" -> Type | "function" -> Function
  | "locals" -> Locals | "void" -> Void | "entry" -> Entry
  | "to" -> To | "const" -> Const | "cast" -> Cast | "un" -> Un
  | "bin" -> Bin | "addr_of" -> Addr_Of | ...
```

For the purpose of the lexer the operator keywords, such as `add`, `sub`, `neg`,... were not defined as tokens themselves, but instead consumed as Idents after the `Un` or `Bin` tokens, since they are not reserved words according to the assignment specification. Every other expected

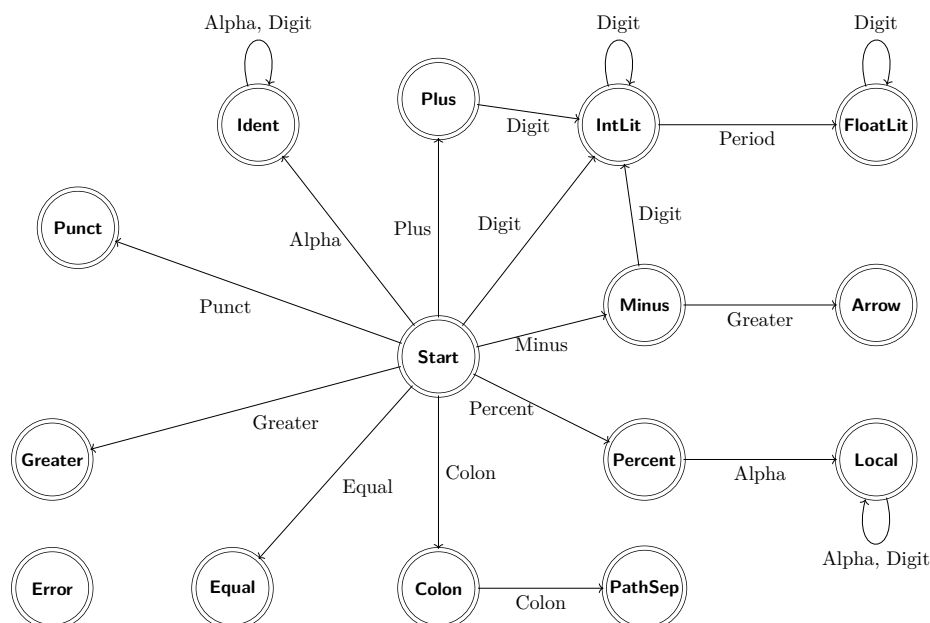
keyword or punctuation defined in the grammar of the ResIR language has its own assigned token, with some having implicit types, such as `Ident` of `string` and `IntLit` of `int`. Notably, `Local` of `string` allows for a `Local` to be tokenized as a single item rather than having to carry a percent token and an `Ident` token into the parser.

The same is true for the `Label` of `string`, but this is not something the DFSA can resolve itself, since something like `bb10` is handled as an `Ident`. Instead this is resolved when mapping states onto tokens, using the `is_numeric` and `is_label` functions, which check if the `Ident` is of the `Label` form `'bb' Digit {Digit}`.

A `chars` type and a `states` type were defined to make the transition table more readable. Rather than the table being full of seemingly random numbers, instead it cleanly indicates what character will cause which transition, with the only added overhead being the simple `map_char`, `st_int`, `ch_int` functions. This then allows for the `map_states` function to assign the correct token to a respective lexeme based on the state that the transition table stopped at, including easy error checking as shown below:

```
let map_states state lexeme line col =
  let err_prefix =
    Printf.sprintf "Lexical Error at Line %d, Col %d: " line col in
  match state with
  | IdentSt -> if is_label lexeme then Label lexeme else map_keywords lexeme
  ...
  (* incomplete token states *)
  | StartSt -> failwith (err_prefix ^ "Empty Token")
  | PercentSt ->
    failwith (err_prefix ^ "Standalone '%' symbol. Expected an identifier.")
  | PlusSt ->
    failwith (err_prefix ^ "Standalone '+' symbol. Expected a literal.
      Did you mean 'add'?")
  ...
```

The only deviation from the supplied grammar in this section is that I decided to allow positive number literals, such as `+45`, so an extra state is needed in the transition table. A benefit of this is that it allows for a more clear error message when something like `22 + 33` is found by the Lexer, as shown in the code above. Below is the DFSA implemented by the transition table in `lexer.ml`, where every missing state transition is mapped onto the Error state:



The Lexer makes use of the above tools to collect tokens by using the transition table to move to the next state until the next state is the `ErrorState`, indicating that the token needs to be consumed as is. If the first character is illegal the `ErrorState` is kept and an invalid fragment error is thrown, otherwise the lexeme is mapped onto the respective token it represents based on the state the DFSA accepted, in the `map_states` function.

This is all done while the current position is tracked using the mutable fields in the `char_pos` type. These allow for the Lexer to have more precise error reporting that shows the user where a lexical error can be found, so for example when the Lexer runs on the following code it will throw the error `Lexical Error at Line 6, Col 15: Standalone '%' symbol. Expected an identifier..`

```
function math_diff() -> i32 {
  bb0:
    %a = const +32;
    %b = const -10;
    %res = const 3.14;
    return %a;%
}
```

Note that this invalid ResIR code, pulled from `lexer_tests/test3_err.resir`, was used to show that the Lexer can only report lexical errors, and not syntactical, since the locals block and entry declaration required by the grammar are not present.

The most important function in `lexer.ml` is the `tokenize` function, which takes as input the source ResIR code read from a file as one continuous string, and returns a list of tokens with their positions (row and column). For example in the above code, the first couple of elements in the list would be `[(Function, 1, 1), (Ident(math_diff), 1, 10), ...]`.

2.2 Task 2 - parser.ml

In order to implement the parser, the first thing that needed to be setup was a type for each item defined in the core grammar of the language, starting with the simplest non-terminals, then moving onto the compound non-terminals composed from them. Those non-terminals that were composed of multiple parts were implemented as tuples or records, with the latter being chosen when one of the fields is optional, such as for the `stmt` type, or when it made the individual parts easier to assign, such as for `block.label`, `block.stmts`, `block.term`. Below are a few notable examples:

```
type path = string list (* ["Custom"; "Struct"; "Point"] *)
type local = Local of string (* "%var" *)
type label = Label of string (* "bb0" *)

type prim_type = | Bool | I32 | I64 | U32 | F64
...
type block = {
  label : label;
  stmts : stmt list;
  term : term;
}
...
```

I then defined simple helper functions for each of these types (`string_of_...`) that prints them as a string for testing purposes. These functions just take inputs of these new types, and based on what type it is operating on prints it back as ResIR syntax. So after running the parser to obtain a variable of type `program`, the `print_parsed` function grabs that `program` variable and converts it back to a continuous ResIR string.

Moving onto actually traversing the token list to parse it, the general strategy was to again define a function for each type (`parse_...`) which start from the most basic types and build up. Each of these functions takes as input the list of tokens and attempts to match it to the expected format of the specific type that it is defined for, and if it succeeds it returns a pair of the newly found element, in the form of the new parser types, and the rest of the token list which has not been parsed yet. In this way a syntax tree is built. Along with these a new function `expect` was used to check the next token, and if the next token is what is expected, then it is dropped and the rest of the token list is returned. This function was necessary to handle punctuation or simple identifiers such as `un` or `bin`, as shown below:

```
let parse_unop tokens =
  (* handling unary operators *)
  let rest1 = expect Lexer.Un tokens in
  match rest1 with
  | (Lexer.Ident "neg", _, _) :: rest2 -> (Neg, rest2)
  | (Lexer.Ident "not", _, _) :: rest2 -> (Not, rest2)

  | (Lexer.Ident bad_op, line, col) :: _ ->
    failwith (Printf.sprintf "Syntax Error at Line %d, Col %d: Expected a
      unary operator (\"neg\" or \"not\") but found \"%s\"." line col bad_op)

  | (t, line, col) :: _ ->
    failwith (Printf.sprintf "Syntax Error at Line %d, Col %d: Expected a
      unary operator (\"neg\" or \"not\")" line col)
  | [] -> failwith "Compiler Bug: Token stream empty without hitting EOF."
```

Note that in `parser.ml` "Lexer." is needed to make a difference between the types defined for the tokens in `lexer.ml`, and the new types defined for the AST.

As is illustrated in the above example as well, detailed error reporting is established in each parse function, with reporting of the exact position of the error possible since each token is wrapped with its row and column index. Also the fact that the operators are not keywords, as mentioned in the previous section, is dealt with here by handling them as Idents, and checking if an invalid operator was inputted.

Some of the parser functions needed to handle 0 or more of a certain type, for example a function's parameters, defined in the grammar as `[Params]`. This was implemented in the form of a loop that checks if another element follows, based on how it starts, or if the element we are currently on is the final one. The below example shows how this is used to parse the parameters of a function by first checking if there are any parameters by looking for a closing parentheses (`RBracket`), then if yes parsing a parameter until the next token is not a comma:

```
let parse_params tokens =
  (* grabbing one param, then looping over any others *)
  match tokens with
  | (Lexer.RBracket, _, _) :: _ -> ([], tokens)
  | _ ->
    let (first_param, rest1) = parse_param tokens in
    let rec params_loop acc loop_tokens =
      match loop_tokens with
      | (Lexer.Comma, _, _) :: loop_tokens1 ->
        let (new_param, loop_tokens2) = parse_param loop_tokens1 in
        params_loop (new_param :: acc) loop_tokens2

      | _ -> (List.rev acc, loop_tokens1)

    in params_loop [first_param] rest1
```

As mentioned in the assignment spec, the grammar was *"easy to parse with one token of lookahead in **almost** all cases"*. The only exceptions I found were in `parse_stmt` to differentiate between Local '=' Rhs and just a destination-less Rhs, such as a void `store` call; and in `parse_term` to differentiate between returning a local and returning nothing, as shown in the below example:

```
let parse_term tokens =
  match tokens with
  ...
  (* if return followed by local, consume both *)
  | (Lexer.Return, _, _) :: (Lexer.Local _, _, _) :: _ ->
    let rest1 = expect Lexer.Return tokens in
    let (local, rest2) = parse_local rest1 in
    (Return (Some local), rest2)

  (* if no local, return none *)
  | (Lexer.Return, _, _) :: rest1 ->
    (Return None, rest1)
  ...
```

These `parse_...` functions culminate in the `parse_program` function which is the most important function of `parser.ml`. This function takes as input a list of tokens (with their positions) generated by the `tokenize` function, and returns a pair of the parsed program, which is a record bundling the extern declarations and the function definition, with any remaining tokens (which is confirmed to be only EOF, since no tokens can follow it).

2.3 Task 3 - type_checker.ml

The type_checker is composed of two main parts, building the environment of the current program and then iteratively checking all the blocks in the environment for semantic errors. This environment is made up of the following parts:

- `extern_types` - a list of struct names (extern paths) and their corresponding types (extern fields)
- `locals` - a list of local names and their types
- `labels` - a list of valid block names
- `funct` - the function currently being checked
- `extern_functions` - a list of function paths and signatures against which calls can be type-checked

While building the environment the following important checks are done: checking for duplicate extern type declarations, duplicate variable declarations, duplicate block labels, the existence of the declared entry block, and if all custom types were declared (jump and cjump target validation happens later). Below is an example for one of these:

```
let build_environment program =
  ...
  (* group params and locals then check they're all unique *)
  let all_locals = program.funct.params @ program.funct.locals in
  let local_names = List.map fst all_locals in
  let unique_local_names = List.sort_uniq compare local_names in
  if List.length local_names <> List.length unique_local_names then
    failwith "Semantic Error: Duplicate local variable or parameter
             declarations.";
  ...
```

Currently the `extern_functions` field in the environment is populated with a hardcoded known function from the tests. This list allows for the parameters in a `call` instruction to be type-checked as well, but it is not necessary to have a function defined here for it be called. If a function not found in `extern_functions` is called, the only difference is that the parameters will not be type checked as the compiler has no way of obtaining the function signature, other than it being hardcoded here. The function signatures had to be hardcoded in this way because the grammar does not provide the syntax to declare external function signatures, only `extern type` to declare struct layouts.

```
let check_statement env stmt =
  match stmt.dest, stmt.rhs with
  ...
| dest_opt, RhsCall (funct_path, args) ->
  let sig_opt =
    if funct_path = env.funct.path then
      (* recursive call signature *)
      Some {args = List.map snd env.funct.params; ret = env.funct.ret_type}
    else
      (* lookup external function *)
      List.assoc_opt funct_path env.extern_functions
  in
  (match sig_opt with
  (* found matching signature, either recursive or pre-existing, so compare
  signature with vars passed to it *)
  | Some callee_sig ->
    let arg_types = List.map (search_local env) args in

    if List.length arg_types <> List.length callee_sig.args then
      failwith (Printf.sprintf "Semantic Error: Arity mismatch in call
        to \"%s\"." (string_of_path funct_path));

    if arg_types <> callee_sig.args then
      failwith (Printf.sprintf "Semantic Error: Argument type mismatch in
        call to \"%s\"." (string_of_path funct_path));

    let rhs_type_opt = match callee_sig.ret with
      Void -> None | RetType t -> Some t in
    is_valid_dest env dest_opt rhs_type_opt

  | None ->
    (* call to an undefined function, first checking that args exist,
    then checking that dest_local exists *)
    List.iter (fun arg -> ignore (search_local env arg)) args;
    match dest_opt with
    | Some dest_local -> ignore (search_local env dest_local)
    | None -> () )
```

Moving onto the way the blocks are checked, the `check_block` function makes use of the `check_statement` and `check_term` functions to confirm that every statement and the terminator are individually valid, raising on the first violation. The `check_statement` function confirms that either it is a void operation, or it is a valid variable declaration or operation. Some of the errors which are checked for include type mismatch when assigning, invalid operator application (such as `neg` to a variable of `Bool` type), invalid type cast (these were derived from C11 type casting rules), and pointer errors in the `MemberPtr`, `Load`, and `Store` operations. The below excerpt includes examples of how some of these errors are raised to the user:

```
let get_rhs_type env expr = (* called as part of the check_statement function *)
  match expr with
```

```

| RhsLocal local ->
  Some (search_local env local)

| RhsCast (local, target_type) ->
  let src_type = search_local env local in
  check_type_exists env.extern_types (RetType target_type);
  if is_valid_cast src_type target_type then Some target_type
  else failwith (Printf.sprintf "Semantic Error: Invalid type cast.
    %s cannot be cast to %s." (string_of_resir_type src_type)
    (string_of_resir_type target_type))

| RhsUn (op, local) ->
  let typ = search_local env local in
  match op, typ with
  | Neg, Prim Bool ->
    failwith "Semantic Error: Cannot apply Neg to local of type Bool.
      Did you mean 'not'?"
  | Neg, Prim _ -> Some typ
  ...

| RhsLoad ptr_local ->
  (match search_local env ptr_local with
  | Ptr inner_type -> Some inner_type
  | _ -> failwith "Semantic Error: Cannot load from a non-pointer type.")
  ...

```

Note that in the above the function `check_type_exists` had to be used outside of environment building to ensure that the cast has a valid target type. This function matches against a variable of type `ret_type`, so it can handle any variables of `resir_type` in the same way, by wrapping them with `RetType`. This implementation avoids code redundancy in having two functions for both `ret_type` and `resir_type`.

```

let rec check_type_exists ext_types ret_typ =
  match ret_typ with
  | Void -> ()
  | RetType typ ->
    match typ with
    ...

```

It was considered for the `is_valid_dest` function to prevent dead-end operations, such as `load %p`, where the value of the pointer is loaded and immediately discarded. However this is something which will later be handled by the optimiser in Task 5, so ultimately I opted to stick as closely to the grammar as possible. Thus these types of operations are allowed, as seen below (the code preventing dead-end operations is left as a comment for reference):

```

let is_valid_dest env dest rhs_type =
  match dest, rhs_type with
  | Some dest_local, Some rhs_type ->
    let dest_type = search_local env dest_local in
    if dest_type <> rhs_type then
      failwith (Printf.sprintf "Semantic Error: Type mismatch.
        Cannot assign type %s to variable %s of type %s."
        (string_of_resir_type rhs_type) (string_of_local dest_local)
        (string_of_resir_type dest_type))

  | None, _ -> () (* void operation with no destination
    or discarding a produced value *)

  | Some dest_local, None ->

```

```

failwith (Printf.sprintf "Semantic Error: Cannot assign a void
    expression to variable %s." (string_of_local dest_local))

(* | None, Some rhs_type ->
failwith (Printf.sprintf "Semantic Error: Expression produces
    a value of type %s but no destination variable was provided."
    (string_of_resir_type rhs_type)) *)

```

Then the `check_term` function confirms that the terminator is valid. If it is a jump statement it confirms that the target labels exist and, additionally for `cjump`, that the condition is of `Bool` type. If it is a return statement, it checks that the variable being returned matches the return type of the function, if it's not void, as seen below:

```

let check_term env term =
  match term with
  | Jump target_label ->
    if not (List.mem target_label env.labels) then
      failwith (Printf.sprintf "Semantic Error: Target of jump \"%s\"
        does not exist." (string_of_label target_label))
    ...
  | Return local_opt ->
    match env.funct.ret_type, local_opt with
    | Void, None -> ()
    | RetType expected_type, Some ret_local ->
      let actual_type = search_local env ret_local in
      if actual_type <> expected_type then
        failwith (Printf.sprintf "Semantic Error: Return type mismatch.
          Expected %s but found %s." (string_of_resir_type expected_type)
            (string_of_resir_type actual_type))
      ...

```

The final important function of `type_checker.ml` is `validate_program` which takes as input a pair where the first element is of program type, builds an environment on it, then checks each of its blocks, by checking all of their statements and their terminators. This function does not output anything if it succeeds, its only goal is to check for all the errors discussed above, and to report one as soon as it finds it.

2.4 Task 4 - code_generator.ml

For the actual C code generation `gen...` functions were defined for each type, similar to how the `parse...` functions were defined for each type, that cleanly map the types that were previously defined in `parser.ml` onto the corresponding C11 code. The below code illustrates this for types, unary operators, and a RHS example of unary operators:

```

let rec gen_type resir_type =
  (* match each ResIR type to its corresponding C-type *)
  match resir_type with
  | Prim Bool -> "bool"
  | Prim I32 -> "int32_t"
  | Prim I64 -> "int64_t"
  ...
...

let gen_unop unop =
  (* matching to C11 operators *)
  match unop with
  | Neg -> "-"
  | Not -> "!"

```



```

...
let gen_rhs rhs =
  (* match RHS to C11 equivalent *)
  match rhs with
  | RhsLocal loc          -> gen_local loc
  | RhsConst lit          -> gen_literal lit
  | RhsCast (loc, typ)    -> Printf.sprintf "(%s)%s"
    (gen_type typ) (gen_local loc)
  (* for example (int32_t(x))* *)
  | RhsUn (op, loc)       -> Printf.sprintf "%s%s"
    (gen_unop op) (gen_local loc)
  (* for example (-x)* *)
...

```

One important design decision I took in this section is to group variables of the same type, so that in the generated C code they can be declared in one line. So for example the ResIR code:

```

locals {
  %zero : i32;
  %one  : i32;
  %sum  : i32;
  ...
}

```

maps cleanly onto

```

int32_t zero, one, sum, ...;

```

However, this had the side effect of changing how pointer types are generated, because simply appending a star to the type when defining a list of variables in this way only makes the first variable a pointer, since the star is bundled with the variable not the type. As a result the sub-function `add_ptr_stars` was defined, which simply goes down the layers of pointers, and appends a star each time a `Ptr` type is found, then groups by the number of stars they have. Thus the following section of ResIR code

```

locals {
  %x : ptr<i32>;
  %y : ptr<i32>;
  %z : ptr<ptr<i32>>;
  ...
}

```

compiles as

```

int32_t *x, *y, ...;
int32_t **z, ...;

```

Also, when a parameter is defined for a function, but all lines using it are removed from that function as a result of the Dead-Code Elimination carried out in Task 5, then the parameter is left 'hanging', which raises a warning since we are compiling with the `-Wextra` flag enabled. An example of this will be shown in the next section. To avoid this warning, the first line of generated C code casts all of the parameters onto void, thus silencing the unused parameter warning. This can be seen in the following excerpt taken from `test1.expected.c`:

```

int32_t Test_int_bool_control(int32_t a, int32_t b, bool flag) {
  (void)a, (void)b, (void)flag;
  ...
}

```

Furthermore, ResIR functions with no parameters are translated into C code with an explicit void parameter list, as seen in tests 3 and 4 below.

The most important function for task 4 in `code_generator.ml` is `gen_program`, which takes a pair where the first element is a program as input, and returns a continuous string of the equivalent, compiled C code by translating the validated AST into C that compiles with `gcc -std=c11 -Wall -Wextra -pedantic generated.c` as required. After finishing task 5 I added the `gen_program_opt` function that does the whole compilation pipeline, from ResIR source code to optimised, compilable C source code.

Now, as mentioned in the assignment specification, the compiler does not generate any C code for the extern types, but instead the struct for these is expected to live in some header, which I have named `extern_types.h`, so each generated C file starts with the line `#include "extern_types.h"`. Also each compiled C file comes with the line `#include <stddef.h>`, since NULL is not provided by the other two headers, and it is part of ResIR grammar.

```
//for test 2
typedef struct Custom_Struct_Point {
    int32_t x, y;
    bool valid;
    struct Custom_Struct_Point* next;
} Custom_Struct_Point;

//for test 3
int32_t Runtime_zero_i32(void) {
    return 0;
}

int64_t Runtime_combine_i64(int64_t a, int64_t b){
    return a + b;
}
```

extern_types.h used for the following tests

Moving onto the examples of input ResIR functions, these were taken from the test cases of `code_generator.ml`, so prior to the `optimise_program` function being run on them, which is why they include some lines of dead code.

The first test goes through each and every unary and binary operator, to confirm that they all compile as required. Compiling this pre-optimisation output produces warnings which Task 5's Dead-Code Elimination removes.

```
function Test::int_bool_control(%a: i32, %b: i32, %flag: bool) -> i32 {
    locals {
        %zero : i32;
        %one : i32;
        %sum : i32;
        %diff : i32;
        %prod : i32;
        %quot : i32;
        %rem : i32;
        %neg : i32;
        %eq : bool;
        %ne : bool;
        %lt : bool;
        %le : bool;
        %gt : bool;
        %ge : bool;
        %notflag : bool;
        %both : bool;
```

```

%or1 : bool;
%or2 : bool;
%or3 : bool;
%cmp_any : bool;
%flag_mix : bool;
%final_cond : bool;
%chosen : i32;
}
entry bb0;

bb0:
%zero = const 0;
%one = const 1;
%sum = bin add %a, %b;
%diff = bin sub %a, %b;
%prod = bin mul %sum, %diff;
%quot = bin div %prod, %one;
%rem = bin mod %quot, %one;
%neg = un neg %diff;
%eq = bin eq %a, %b;
%ne = bin ne %a, %b;
%lt = bin lt %a, %b;
%le = bin le %a, %b;
%gt = bin gt %a, %b;
%ge = bin ge %a, %b;
%notflag = un not %flag;
%both = bin and %flag, %notflag;
%or1 = bin or %eq, %ne;
%or2 = bin or %lt, %le;
%or3 = bin or %gt, %ge;
%cmp_any = bin or %or2, %or3;
%flag_mix = bin or %both, %notflag;
%final_cond = bin or %or1, %cmp_any;
cjump %final_cond, bb1, bb2;

bb1:
%chosen = %neg;
jump bb3;

bb2:
%chosen = %rem;
jump bb3;

bb3:
return %chosen;
}

```

test1.resir

```

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "extern_types.h"

int32_t Test_int_bool_control(int32_t a, int32_t b, bool flag) {
    (void)a, (void)b, (void)flag;
    int32_t zero, one, sum, diff, prod, quot, rem, neg, chosen;
    bool eq, ne, lt, le, gt, ge, notflag, both, or1, or2, or3, cmp_any,
        flag_mix, final_cond; //wrapped around to fit in this text box
    goto bb0;

bb0:
    zero = 0;

```

```

    one = 1;
    sum = a + b;
    diff = a - b;
    prod = sum * diff;
    quot = prod / one;
    rem = quot % one;
    neg = -diff;
    eq = a == b;
    ne = a != b;
    lt = a < b;
    le = a <= b;
    gt = a > b;
    ge = a >= b;
    notflag = !flag;
    both = flag && notflag;
    or1 = eq || ne;
    or2 = lt || le;
    or3 = gt || ge;
    cmp_any = or2 || or3;
    flag_mix = both || notflag;
    final_cond = or1 || cmp_any;
    if (final_cond) goto bb1; else goto bb2;

bb1:
    chosen = neg;
    goto bb3;

bb2:
    chosen = rem;
    goto bb3;

bb3:
    return chosen;
}

```

test1.expected.c

The second test checks pointers, and possible uses of the `member_ptr`, `load`, and `store` keywords.

```

extern type Custom::Struct::Point {
    x : i32;
    y : i32;
    valid : bool;
    next : ptr<Custom::Struct::Point>;
}

function Test::move_point(%p: ptr<Custom::Struct::Point>, %dx: i32, %dy: i32)
    -> void { //wrapped around to fit in this text box
    locals {
        %xptr : ptr<i32>;
        %yptr : ptr<i32>;
        %vptr : ptr<bool>;
        %nptr : ptr<ptr<Custom::Struct::Point>>;
        %x : i32;
        %y : i32;
        %newx : i32;
        %newy : i32;
        %ok : bool;
        %nullp : ptr<Custom::Struct::Point>;
        %is_root : bool;
        %local_copy : i32;
        %copy_ptr : ptr<i32>;
    }
}

```

```

}
entry bb0;

bb0:
    %nullp = const null;
    %is_root = bin ne %p, %nullp;
    %xptra = member_ptr %p, x;
    %yptra = member_ptr %p, y;
    %vptra = member_ptr %p, valid;
    %nptra = member_ptr %p, next;
    %x = load %xptra;
    %y = load %yptra;
    %newx = bin add %x, %dx;
    %newy = bin add %y, %dy;
    store %xptra, %newx;
    store %yptra, %newy;
    %ok = const true;
    store %vptra, %ok;
    %local_copy = %newx;
    %copy_ptr = addr_of %local_copy;
    store %copy_ptr, %newy;
    cjump %is_root, bb1, bb2;

bb1:
    store %nptra, %nullp;
    return;

bb2:
    return;
}

```

test2.resir

```

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "extern_types.h"

void Test_move_point(Custom_Struct_Point* p, int32_t dx, int32_t dy) {
    (void)p, (void)dx, (void)dy;
    int32_t *xptra, *yptra, *copy_ptr;
    bool *vptra;
    Custom_Struct_Point **nptra;
    int32_t x, y, newx, newy, local_copy;
    bool ok, is_root;
    Custom_Struct_Point *nullp;
    goto bb0;

bb0:
    nullp = NULL;
    is_root = p != nullp;
    xptra = &(p->x);
    yptra = &(p->y);
    vptra = &(p->valid);
    nptra = &(p->next);
    x = *xptra;
    y = *yptra;
    newx = x + dx;
    newy = y + dy;
    *xptra = newx;
    *yptra = newy;
    ok = true;
    *vptra = ok;
}

```

```

    local_copy = newx;
    copy_ptr = &local_copy;
    *copy_ptr = newy;
    if (is_root) goto bb1; else goto bb2;

bb1:
    *nptr = nullptr;
    return;

bb2:
    return;
}

```

test2.expected.c

The third test checks casts and calls, including a call to an external function `Runtime::combine_i64` whose signature is not hardcoded into the compiler's environment. This demonstrates how the type checker gracefully bypasses signature validation for unknown external functions, deferring to the C compiler via `extern_types.h`.

```

function Test::calls_casts_numeric() -> i64 {
  locals {
    %i : i32;
    %wide : i64;
    %u : u32;
    %fromu : i64;
    %f : f64;
    %negf : f64;
    %sumf : f64;
    %halff : f64;
    %fromf : i32;
    %fromsum : i32;
    %fromf64 : i64;
    %fromsum64 : i64;
    %call0 : i32;
    %call1 : i64;
    %tmp : i64;
    %ok : bool;
    %ret : i64;
  }
  entry bb0;

bb0:
  %i = const -7;
  %wide = cast %i to i64;
  %u = const 42;
  %fromu = cast %u to i64;
  %f = const 3.5;
  %negf = un neg %f;
  %sumf = bin add %f, %negf;
  %halff = bin div %f, %f;
  %fromf = cast %halff to i32;
  %fromsum = cast %sumf to i32;
  %fromf64 = cast %fromf to i64;
  %fromsum64 = cast %fromsum to i64;
  %call0 = call Runtime::zero_i32();
  %call1 = call Runtime::combine_i64(%wide, %fromu);
  %tmp = bin add %call1, %fromf64;
  %ok = bin ge %tmp, %fromsum64;
  cjump %ok, bb1, bb2;

bb1:
  %ret = bin add %tmp, %fromu;
}

```

```

    return %ret;

bb2:
    %ret = cast %call0 to i64;
    return %ret;
}

```

test3.resir

```

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "extern_types.h"

int64_t Test_calls_casts_numeric(void) {
    int32_t i, fromf, fromsum, call0;
    int64_t wide, fromu, fromf64, fromsum64, call1, tmp, ret;
    uint32_t u;
    double f, negf, sumf, halff;
    bool ok;
    goto bb0;

bb0:
    i = -7;
    wide = (int64_t)i;
    u = 42;
    fromu = (int64_t)u;
    f = 3.5;
    negf = -f;
    sumf = f + negf;
    halff = f / f;
    fromf = (int32_t)halff;
    fromsum = (int32_t)sumf;
    fromf64 = (int64_t)fromf;
    fromsum64 = (int64_t)fromsum;
    call0 = Runtime_zero_i32();
    call1 = Runtime_combine_i64(wide, fromu);
    tmp = call1 + fromf64;
    ok = tmp >= fromsum64;
    if (ok) goto bb1; else goto bb2;

bb1:
    ret = tmp + fromu;
    return ret;

bb2:
    ret = (int64_t)call0;
    return ret;
}

```

test3.expected.c

And lastly the fourth test is a very simple minimal function to check that this is handled safely too.

```

function Test::minimal_empty_void() -> void {
    locals {
    }
    entry bb0;

    bb0:
        return;
}

```

```
}
```

test4.resir

```
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "extern_types.h"

void Test_minimal_empty_void(void) {
    goto bb0;

bb0:
    return;
}
```

test4.expected.c

2.5 Task 5 - optimiser.ml

The goal of the `optimise_program` function in the `optimiser.ml` file is to implement Dead Code Elimination by using a backward Liveness Analysis. It does this by identifying variables that are defined, but then never used, to safely delete their assignment statements and make the generated C code lighter.

This analysis is made possible by the `block_gen_kill` function, which takes a block as input to compute the following sets:

- **Gen set:** Variables read in the block before being written to.
- **Kill set:** Variables written to in the block.

These sets are used to calculate the following fixpoint equations:

$$LiveIn(b) = Gen(b) \cup (LiveOut(b) \setminus Kill(b))$$

$$LiveOut(b) = \bigcup_{s \in succ(b)} LiveIn(s)$$

I made use of OCaml's `StringSet` module to compute these sets, and the `StringMap` module to store them efficiently associated with the block they were computed for. Then the sub-function `worklist_loop` in the `compute_liveness` function forms a queue of the blocks passed to the function. It iteratively pops a block from this queue, recalculates $LiveOut(b)$ by taking the union of the In sets of all its successors, then computes the new $In(b)$ with the equations described above. If this new $In(b)$ set is different from its previous state, then the block's predecessors are pushed back onto the worklist. This process is repeated until $In(b)$ stops changing for all the blocks, and so a fixpoint is reached.

```
let compute_liveness blocks =
  (* start by building all required maps to run worklist *)
  let (succ_map, pred_map) = build_cfg_maps blocks in

  let (gen_map, kill_map, in_map, out_map, initial_worklist) =
    List.fold_left map_init_block (StringMap.empty, StringMap.empty,
      StringMap.empty, StringMap.empty, []) blocks

  in
  let rec worklist_loop wl current_in current_out =
    match wl with
    | [] -> (current_in, current_out) (* worklist is empty so we
      have reached the fixpoint *)
```



```

| b :: rest_wl ->
  (* compute LiveOut[B] (the union of LiveIn[S] for each succ S) *)
  let succs =
    match StringMap.find_opt b succ_map with
    | Some l -> l
    | None    -> []
  in

  let new_out_b = List.fold_left (fun acc succ ->
    let in_succ =
      match StringMap.find_opt succ current_in with
      | Some s -> s
      | None -> StringSet.empty in
      StringSet.union acc in_succ) StringSet.empty succs in

  (* compute LiveIn[B] (gen[B] U (LiveOut[B] \ kill[B])) *)
  let gen_b = StringMap.find b gen_map in
  let kill_b = StringMap.find b kill_map in
  let old_in_b = StringMap.find b current_in in

  let new_in_b = StringSet.union gen_b (StringSet.diff new_out_b kill_b) in

  let new_out = StringMap.add b new_out_b current_out in
  let new_in = StringMap.add b new_in_b current_in in

  (* fixpoint check *)
  if StringSet.equal old_in_b new_in_b then
    worklist_loop rest_wl new_in new_out

  else
    (* if In[B] changed re-evaluate non-duplicate predecessors *)
    let preds =
      match StringMap.find_opt b pred_map with
      | Some l -> l
      | None -> []
    in

    let next_wl = List.fold_left (fun acc p ->
      if List.mem p acc then acc else p :: acc ) rest_wl preds
    in

    worklist_loop next_wl new_in new_out
in

worklist_loop initial_worklist in_map out_map

```

Once these liveness sets are computed for each block, the Dead Code Elimination is handled in the `optimise_block` function, which performs a bottom-up traversal of each statement in the block, and saves variables in its *LiveOut* set, starting from the terminator. If an assignment is found with a destination that is not currently live, then that means that this assignment is unused. Also, destination-less statements are treated as dead by default, but `safe_delete` protects them from erroneous deletion. However this is not the only condition for removal of that statement, because some operations, like `store` operations, may have an effect on the memory state outside the currently running program. This is checked by the aforementioned `safe_delete` function, and if a statement is deemed to be safe to delete and dead then it is dropped and the walk moves to the next statement above it. Otherwise the statement destination is removed from the live set, and its uses are added instead, with the statement being saved.

```

let safe_delete rhs =
  (* call and store have other in-memory side effects so they are

```

```

    never safe to delete *)
match rhs with
| RhsCall _ | RhsStore _ -> false
| _ -> true

let optimise_block block live_out =
  (* add terminator uses manually *)
  let initial_live = StringSet.union live_out (extract_term_uses block.term) in

  (* walk through block statements checking if each is dead code *)
  let rec walk_stmts current_live acc_stmts =
    match stmts with
    | [] -> acc_stmts
    | stmt :: rest ->
      let is_dead =
        match stmt.dest with
        | Some loc -> not (StringSet.mem (string_of_local loc) current_live)
        | None -> true
      in

      if is_dead && safe_delete stmt.rhs then
        (* drop stmt if it is dead and safe to delete *)
        walk rest current_live acc_stmts
      else
        (* if not dead or unsafe, remove dest from live set and add
           its uses to live set and keep it *)
        let new_live_without_def =
          match stmt.dest with
          | Some loc -> StringSet.remove (string_of_local loc) current_live
          | None -> current_live
        in
        let uses = extract_rhs_uses stmt.rhs in
        let new_live = StringSet.union new_live_without_def uses in
        walk rest new_live (stmt :: acc_stmts)
    in
  (* working bottom-up *)
  let optimised_stmts = walk (List.rev block.stmts) initial_live [] in
  {block with stmts = optimised_stmts}

```

Finally, the list of locals for the function is also cleaned up by comparing it with the list returned by the `extract_funcnt_survivors` function. So any variables whose assignment statements were fully deleted during the DCE pass are not printed in the final C output, avoiding any unused variable flags, unless of course these statements are `store` or `call` instructions as discussed previously.

```

let extract_funcnt_survivors blocks =
  List.fold_left (fun acc block ->
    StringSet.union acc (extract_block_survivors block)) StringSet.empty blocks

```

So the most important function in `optimiser.ml` is the `optimise_program` function that takes a pair where the first element is of program type as input, and returns a pair where the first element is the program with optimised blocks and locals, and so an optimised function.

As an example the code below is what is generated when the AST is optimised before it is generated, which in comparison with `test1.expected.c` above has much fewer lines, since most of these were just placed there to test operators. It's important to note in the below that my implementation successfully eliminates those variables that become unused by the initial elimination of dead code, such as `notflag`, which is only read by the now-dead `flag_mix`. Also in the below example the `flag` parameter becomes dead, and since it was cast to `void`, no warnings are thrown, as discussed in Task 4.

```

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "extern_types.h"

int32_t Test_int_bool_control(int32_t a, int32_t b, bool flag) {
    (void)a, (void)b, (void)flag;
    int32_t one, sum, diff, prod, quot, rem, neg, chosen;
    bool eq, ne, lt, le, gt, ge, or1, or2, or3, cmp_any, final_cond;
    goto bb0;

bb0:
    one = 1;
    sum = a + b;
    diff = a - b;
    prod = sum * diff;
    quot = prod / one;
    rem = quot % one;
    neg = -diff;
    eq = a == b;
    ne = a != b;
    lt = a < b;
    le = a <= b;
    gt = a > b;
    ge = a >= b;
    or1 = eq || ne;
    or2 = lt || le;
    or3 = gt || ge;
    cmp_any = or2 || or3;
    final_cond = or1 || cmp_any;
    if (final_cond) goto bb1; else goto bb2;

bb1:
    chosen = neg;
    goto bb3;

bb2:
    chosen = rem;
    goto bb3;

bb3:
    return chosen;
}

```

test1.expected.c - optimised

So for the above code a sample of the facts derived from its CFG are:

```

edge(bb0, bb1).
edge(bb0, bb2).
edge(bb1, bb3).
edge(bb2, bb3).

def(s_bb0_0, %one).
def(s_bb0_1, %sum).
def(s_bb0_2, %diff).
...

use(s_bb0_1, %a).
use(s_bb0_1, %b).
use(term_bb0, %final_cond).

```

And the following liveness rules are implemented on them by the `worklist_loop` sub-function, which computes the same fixpoint via backward dataflow iteration over the CFG.

```
live_in(B, X)  :- gen(B, X).
live_in(B, X)  :- live_out(B, X), not kill(B, X).
live_out(B, X) :- edge(B, S), live_in(S, X).
dead_assign(B, X) :- def_in_block(B, X), not live_out(B, X).
```

So the derived `dead` facts that were used to drop the corresponding assignment statements for `notflag` and `flag_mix` (and as a result unrefenced) are:

```
dead_assign(bb0, %notflag).
dead_assign(bb0, %flag_mix).
```

3 Testing Suite

The test suite consists of 5 `.ml` files, one for each task described above, and 4-6 test cases for each. These all follow the same format of comparing the output generated by the input ResIR code in `testx.resir` with some `testx.expected`, with the last two being of the form `testx.expected.c`, since they are generating C code. Additionally, `optimiser_test.ml` also compiles the code using the required flags to check that no warnings are thrown.

The automated test suite can be run with `dune test`, after building the project using `dune build`, and alternatively if one wishes to compile any ResIR code, I have also prepared the `main.ml` file in `bin` to attempt to generate compilable C code for any input ResIR file, by running the following command: `dune exec ./bin/main.exe -- <file_path>`. Below are brief descriptions for what each test is testing for:

Test	Testing Scope
test1.resir	Handles consts, operators, and jumps
test2.resir	Handles extern type, ptr, and other instructions
test3_err.resir	Throws an expected Ident error
test4_err.resir	Throws an invalid fragment error
test5_err.resir	Throws an unexpected '+' error

lexer_tests

Test	Testing Scope
test1.resir	Handles consts, operators, and jumps
test2.resir	Handles extern type, ptr, and other instructions
test3_err.resir	Throws a missing semicolon error
test4_err.resir	Throws a missing label error
test5_err.resir	Throws a missing terminator error
test6_err.resir	Throws a missing function definition error

parser_tests

Test	Testing Scope
test1_err.resir	Throws an invalid jump target error
test2_err.resir	Throws an invalid entry block error
test3_err.resir	Throws an invalid operator application error
test4_err.resir	Throws a type mismatch error
test5_err.resir	Throws a duplicate block label error
test6_err.resir	Throws a bad member_ptr error (since it points to a primitive type)

type_check_tests

Test	Testing Scope
test1.resir	Handles all operators
test2.resir	Handles pointers and member_ptr, load, and store
test3.resir	Handles casts and calls to external functions
test4.resir	Handles a simple minimal function

code_gen_tests

Test	Testing Scope
test1.resir	Eliminates many dead assignments
test2.resir	Eliminates many dead assignments (<code>calc1</code> , <code>calc2</code>), but not store instructions
test3.resir	Eliminates many dead assignments (<code>calc1</code> , <code>calc2</code>), but not destination-less call instructions
test4.resir	Eliminates load statements with no destinations

optimiser_tests

4 Limitations and Future Improvements

The compiler successfully implements the full pipeline required to translate ResIR into optimized, C11-compliant code. However, due to the scope of the language and the architecture of the passes, there are a few limitations.

- **Hardcoded External Function Signatures:** During the semantic analysis phase, the type checker requires external function signatures to validate the arguments passed to `call` instructions. However, because the core ResIR grammar only provides syntax for `extern type` declarations (struct layouts) and not external function signatures, these currently have to be hardcoded into the compiler's `extern_functions` environment. A future improvement would be to extend the ResIR grammar, lexer, and parser to recognize an `extern function` keyword, allowing developers to declare signatures directly in the source file.
- **Pointer Analysis when Optimising:** The optimiser relies on strict register liveness and lacks a complex analysis over the pointer type. Passing variables by reference introduces potential side effects that this simple pass might miss if multiple of them refer to the same location. If a more aggressive optimiser is needed that limits memory use as much as possible, then a more complex analysis pass would be needed.
- **Error Recovery:** Currently when an error is encountered compilation is halted immediately and it is reported. A future improvement would be to skip over that section, and continue compilation so that multiple errors, even of different types, can be identified and addressed in one run.

Overall, the compiler achieves its primary goal of safely parsing, semantically validating, optimizing, and cleanly translating ResIR into C code, while establishing a modular foundation that can easily accommodate the improvements listed above.